MULTI-CORE PROGRAMMING

Exam Notes: Chapter 1 + Chapter 2

Source: *Multi-Core Programming: Increasing Performance through Software Multi-threading*, Shameem Akhter and Jason Roberts, Intel Press, 2006.

Chapter 1: Introduction to Multi-Core Architecture
Chapter 2: System Overview of Threading

How to use these notes

- Every point is taken from the textbook only. Nothing external has been added.
- Read the **Quick Revision Sheet** (Part C) the night before; use **Part D** (practice questions) to self-test.
- Yellow boxes = must-remember / exam favourites. Green boxes = formulas and worked examples. Red boxes = common mistakes and traps.

Contents

Part	Content
A	Chapter 1: Introduction to Multi-Core Architecture
B	Chapter 2: System Overview of Threading
C	Quick revision sheet (definitions, formulas, comparison tables)
D	Practice questions with model answers

PART A: CHAPTER 1 - Introduction to Multi-Core Architecture

1.1 Historical background

- **1945:** John von Neumann (with J. Presper Eckert and John Mauchly) wrote a memo proposing the **EDVAC** (Electronic Discrete Variable Automatic Computer). He suggested the **stored-program model**.
- **Von Neumann architecture:** a program is a sequence of instructions stored sequentially in memory, executed one after another in a **linear, single-threaded** fashion.
- **1960s:** time-sharing operating systems on mainframes introduced **concurrent program execution**. Multiple users could submit jobs at once; each program believed it was the only process in the system. The OS allocated CPU time. Concurrency existed at the **process level**.
- **Early PCs:** standalone, single-user OS, one program at a time, text interfaces, straight-line execution.
- **Later:** exponential growth in performance led to GUIs (multiple programs in the same environment) and pervasive networking. This raised **user expectations**: send e-mail while streaming Internet radio, quick/responsive platform, fast application start, background tasks (e.g. auto-save) with minimal disruption. These are the problems facing today's software developers.

1.2 Motivation for concurrency in software

The streaming multimedia example (baseball game over the Internet)

- **End-user view:** simple, like broadcast TV; continuous uninterrupted delivery.
- **Reality, client side:** download streaming video, decompress/decode it, draw it on the display; handle accompanying audio and send it to the sound card; meanwhile the OS may run a virus scan or other periodic system tasks.
- **Reality, server side:** receive the original broadcast, encode/compress in near real time, send to potentially hundreds of thousands of clients.
- The system designer sees many **independent subsystems operating in parallel**. Decomposing the problem into disparate parts makes each isolated task manageable. (Footnote: 'independently' is used loosely; interdependencies must still be managed.)

Definition

Concurrency in software is a way to manage the sharing of resources used at the same time.

Why concurrency is important (two reasons)

- **1. Efficient use of system resources.** Efficient resource utilisation is key to maximising performance. Unnecessary dependencies between components drastically lower performance.
  - Naive serial client: (1) wait for data from network, (2) uncompress, (3) decode, (4) send to video/audio hardware. **Highly inefficient**: the system is idle while waiting for network data.
  - Better: **stage the work**. While waiting for the next frame to arrive from the network, decode the previous frame on the CPU.
- **2. Many problems lend themselves to simple concurrent implementations.** Concurrency is an abstraction for naturally parallel algorithms/applications.
  - Example: **FTP server**. Single-threaded design must track state for every connection. Multi-threaded design: **one thread per connection**, connection state managed by that thread: simpler and easier to maintain.

EXAM FAVOURITE: Concurrent vs Parallel

- **Parallel:** multiple software threads run **simultaneously on different hardware resources / processing elements**. Multiple threads make progress at the same moment.
- **Concurrent:** execution of threads is **interleaved onto a single hardware resource**. All active threads are ready to execute but only **one** makes progress at any point in time.
- To have parallelism you must have concurrency exploiting multiple hardware resources. (Parallel implies concurrent, but not vice versa.)

1.3 Parallel computing platforms

Hardware must support simultaneous execution of multiple threads. Architectures are classified along two dimensions: (1) number of **instruction streams** processed at one time, (2) number of **data streams** processed at one time. This is **Flynn's taxonomy (Flynn, 1972)**.

Class	Meaning	Notes / examples from book
SISD	Single instruction, single data	Traditional sequential computer; no hardware parallelism; one data stream per clock cycle. Original IBM PC, older mainframes, 8-bit home computers such as the Commodore 64 (early 1980s).
MISD	Multiple instruction, single data	Multiple instruction streams on one data stream. Mostly a **theoretical model**, not a practical mass-produced platform.
SIMD	Single instruction, multiple data	One instruction stream processes multiple data streams at once. Useful for DSP, image processing, audio/video multimedia. Originally array/vector processors such as the **Cray-1**. Intel: **MMX, SSE, SSE2, SSE3** (multiple data elements per clock; stored in floating-point registers). PowerPC: **AltiVec**.
MIMD	Multiple instruction, multiple data	Executes multiple instruction streams, each on a separate independent data stream. **Most common parallel platform today.** Multi-core platforms such as the Intel Core Duo are MIMD.

- Modern machines are SIMD or MIMD, so developers can exploit **data-level** and **task-level** parallelism.

1.4 Parallel computing in microprocessors

Moore's law

- **1965, Gordon Moore:** number of transistors available to manufacturers would **double approximately every 18 to 24 months**. It has guided designers for ~40 years.
- **Common misconception:** that Moore's law predicts CPU clock frequency. Clock speed has tended to follow it, but viewing it that way limits designers; extra transistors can be used in less obvious ways.

Instruction-Level Parallelism (ILP)

- Also called **dynamic or out-of-order execution**. The CPU reorders instructions in an optimal way to **eliminate pipeline stalls**.
- Goal: increase number of instructions executed per clock cycle. Needs multiple **independent** instructions. In in-order execution, dependencies limit available instructions.
- Scheduling is done by the processor itself; parallelism is at hardware level and **transparent to the software developer**.
- A processor able to execute multiple instructions in one clock cycle is a **super-scalar** processor.

Approaches to thread-level parallelism (TLP)

Approach	How it works	Limitation / trade-off
Time-sliced (pre-emptive) multitasking OS	Interleaves execution of multiple threads; hides I/O latencies.	**No parallel execution**: only one instruction stream runs on a processor at one time.
Multiprocessor (more physical processors)	True parallel execution; threads/processes run simultaneously on multiple processors.	**Increases overall system cost.**
Simultaneous multi-threading (SMT) / Hyper-Threading (HT)	Duplicate the architecture state to create **logical processors**; execution resources are shared.	Threads share execution resources; microarchitecture decides how to interleave. Not true parallelism.
Multi-core (chip multiprocessing, CMP)	Two or more execution cores on one processor die/package, each with own execution and architectural resources.	Cores may or may not share a large on-chip cache. Can be combined with SMT.

Thread and processor definitions

- **Thread:** basic unit of CPU utilisation. Contains a **program counter** (points to current instruction), **CPU state** information for the thread, and other resources such as a **stack**.

- **Physical processor** consists of: **architecture state** (general-purpose CPU registers and interrupt controller registers), caches, buses, execution units, branch prediction logic.
- To define a thread, **only the architecture state is required**. So a **logical processor** can be created by **duplicating the architecture state**; execution resources are shared. This technique = **SMT**.
- **HT Technology** = Intel's SMT. Makes one processor appear, to software, as multiple logical processors; OS and applications schedule threads to logical processors as on a multiprocessor system. When one thread stalls (cache miss, branch misprediction) another makes progress.
- **Multi-core processors** use **chip multiprocessing (CMP)**. They are essentially two individual processors on a single die. Combined with SMT, number of logical processors = **2 x number of execution cores** (for 2 threads per core).

Processor architectures in Figure 1.4 (be able to name/draw)

(A) Single core, (B) Multiprocessor, (C) Hyper-Threading Technology, (D) Multi-core, (E) Multi-core with shared cache, (F) Multi-core with Hyper-Threading Technology.

1.5 Differentiating multi-core architectures from Hyper-Threading Technology

Aspect	Hyper-Threading (HT) Technology	Multi-core
Resources	Parts of the one processor shared between threads, other parts duplicated. The key shared resource is the **execution engine**.	Two or more **independent execution cores** in one package; each thread has a hardware execution environment **entirely to itself**.
Execution	Engine works on both threads at the same time, executing one thread's instructions on resources the other is not using; **interleaves instructions in the pipeline**. Not parallel.	Truly **parallel** execution of each thread.
Gain mechanism	**Latency hiding**. If one thread waits (disk read, keyboard), the other takes over all processor resources without OS task switching.	Real parallelism.
Typical gain	In certain applications on average about **30% increase in throughput** (about **1.3x** the instructions of a single thread).	Theoretically close to **100% (2x)** for a dual-core system.
Dependence	Gains vary by application and platform. If cache optimisation already eliminated memory latencies, HT may yield **no gain**.	Needs well-threaded software.

- HT does **not** attempt to deliver multi-core performance. It is a facility by which the programmer may use idle CPU resources to accomplish more work. **Combined with multi-core** it gives powerful optimisation opportunities and substantially higher throughput.

1.6 Multi-threading on single-core versus multi-core platforms

Threads have existed for decades, but there are important considerations when targeting multi-core:

Consideration 1: Optimal performance requires using threads to partition workloads

- On single-core, threads are mostly used to improve **user responsiveness**, e.g. spawn a thread for a database query or disk access rather than block the UI; the scheduler schedules the UI control loop and data task separately. Developers rely on straight-line instruction throughput improvements.
- **Limitation on single-core:** only interleaving, not simultaneous execution, so performance gains are limited. Threads are mainly an **abstraction for hiding latency**.
- **On multi-core:** threads do not have to wait for one resource; they run independently on separate cores. Example: two threads both wanting a **shift operation**: with one 'shifter unit' they cannot run in parallel; with two cores there are two shifter units.
- Multi-core allows intelligently partitioning different workloads on different cores.

Consideration 2: Different design considerations (assumptions valid on single-core may fail)

- **(a) Memory caching and false sharing.** Each core may have its own cache and the caches may be out of sync. Two threads on a dual-core read/write neighbouring locations that lie in the **same cache line** (cache

works on principle of locality). The memory system may mark the line invalid even though the data a thread wants has not changed. This is **false sharing**. On single-core there is one shared cache, so no synchronisation issue.

- **(b) Thread priorities.** A developer might assume a higher-priority thread always runs without interference from a lower-priority one. True on single-core (scheduler will not yield the CPU to the lower-priority thread). On multi-core, the scheduler may place both threads on separate cores so **both run simultaneously**. Code relying on the assumption becomes **unstable** on multi-core and multi-processor systems.

Footnote to remember: multi-core cache designs

- Some multi-core CPUs **share** the on-chip cache between execution units; some give a **dedicated** cache per core; others are **hybrid** (layers dedicated to a core and layers shared by all).
- For the performance discussion in this chapter the book assumes a **dedicated cache per core**.

1.7 Understanding performance

Speedup

Speedup characterises how much faster a program runs when parallelised. It is the ratio of elapsed run time of the **best sequential algorithm** to the elapsed run time of the **parallel implementation**:

Formula: Speedup

- **Speedup(n_t) = Time$_{\text{best sequential algorithm}}$ / Time$_{\text{parallel implementation}}$($n_t$)**
- n_t = number of physical threads used in the parallel implementation.

Amdahl's Law (Gene Amdahl, 1967)

- Gives the **maximum theoretical performance benefit** of a parallel solution relative to the best-case serial solution.
- Starting statement: speedup is a function of the **fraction of the program accelerated** and **by how much** it is accelerated.

General form and its example

- **Speedup = 1 / [(1 - Fraction$_{\text{enhanced}}$) + (Fraction$_{\text{enhanced}}$ / Speedup$_{\text{enhanced}}$)]**
- Example from the book: speed up **half** the program by 15%: Speedup = 1 / [(1 - 0.50) + (0.50/1.15)] = 1 / (0.50 + 0.43) = **1.08**, an 8% increase. If half the program is improved by 15%, the whole program improves by half that amount.

Equation 1.1: Amdahl's Law for parallel programs

- **Speedup = 1 / [S + (1 - S)/n]**
- **S** = time spent executing the **serial portion** of the parallelised version; **n** = number of processor cores. The numerator 1 assumes the best sequential algorithm takes 1 unit of time.
- If n = 1, there is **no speedup**.
- Book's examples (half the work parallelised): dual-core: 1/(0.5 + 0.5/2) = 1/0.75 = **1.33** (33% speed-up; run time is 75% of original). 8-core: 1/(0.5 + 0.5/8) = 1/0.5625 = **1.78**.

Equation 1.2: Upper bound (n = infinity)

- **Speedup = 1 / S**
- Assumes cores are perfectly scalable. The max benefit from parallelising part of the code is limited by the serial portion.
- Example: if **10%** of the application is serial, the maximum speedup is **10x**, regardless of the number of cores.
- Endlessly increasing cores only affects the parallel part of the denominator. If a program is only 10% parallelised, the maximum theoretical benefit is that it runs in **90% of sequential time**.

First corollary of Amdahl's Law

EXAM FAVOURITE

- **Decreasing the serialised portion (by increasing the parallelised portion) is of greater importance than adding more processor cores.**
- Worked example (30% parallelised, dual-core): **doubling the number of cores** reduces run time from **85%** to **77.5%** of serial time; **doubling the amount of parallelised code** reduces run time from **85%** to **70%**. (Figure 1.7)
- Only when a program is **mostly parallelised** does adding processors help more than parallelising the remaining code. There are hard limits on how much code can be serialised and on how many extra cores make a difference.

Amdahl's Law with overhead (realistic multi-core)

Formula

- **Speedup = 1 / [S + (1 - S)/n + H(n)]**
- **H(n)** = overhead of adding threads; assumes best serial algorithm runs in one time unit. Overhead is **not linear** on a good parallel machine.
- Overhead has two parts: **(1) actual operating-system overhead** and **(2) inter-thread activities** such as synchronisation and other communication between threads.
- If overhead is large enough it offsets the parallel benefit; speedup can become **less than 1**, i.e. threading has **slowed** performance versus single-threaded. This is common in poorly architected multi-threaded applications. So the overhead introduced by threading must be kept to a minimum.

Amdahl's Law applied to Hyper-Threading Technology

- On HT processors, certain resources are shared between threads, which directly affects the maximum benefit. Assume about a **30% gain** from HT (performance improves 30% over the time for a single processor to run both threads).
- By contrast, a **quad-core** platform with each core fully dedicated to its thread could in theory give up to **4x** (second, third, fourth cores give a 300% boost). In practice less, due to overhead and non-parallelisable code.
- Inside an HT processor each thread runs **more slowly** than if it had the whole processor. HT is **not a replacement** for multi-core since many resources (e.g. execution units) are shared. The book assumes each thread runs approximately **one-third slower**.

Formula (HT version)

- **Speedup_{HTT} = 1 / [S + 0.67 ((1 - S)/n) + H(n)]**
- n = number of **logical processors**. H(n) is determined **empirically** and varies by application. Represents typical speed-up on cores with HT.

Gustafson's Law (growing returns)

- Based on Amdahl's work, the viability of massive parallelism was questioned for years. In the **late 1980s at Sandia National Lab**, impressive **linear speedups** were observed in three practical applications on a **1,024-processor hypercube** (Gustafson 1988), even when Amdahl's Law predicted otherwise.

Assumptions built into Amdahl's Law that may not hold in reality:

- **1.** Assumes the best serial algorithm is strictly limited by CPU cycle availability. Not always so: a multi-core processor with a separate cache per core can keep more of the data set in cache, **reducing memory latency**.
- **2.** Assumes the serial algorithm is the best possible solution. Some problems have a more efficient parallel solution; **fewer computational steps** may be needed.
- **3. (The biggest weakness)** Assumes **problem size stays the same** as cores increase. In practice, when given more computing resources, the **problem grows to meet the resources**; more often the **run time is constant**.

Scaled speedup (Gustafson's Law), developed by E. Barsis

- **Scaled speedup = N + (1 - N) * s**
- N = number of processor cores; s = ratio of time spent in the **serial** part of the program to total execution time.
- The speedup is **linear** in N. Shown equivalent to Amdahl's Law (**Shi 1996**), but offers a more realistic view of the potential of parallel computing on multi-core processors.

	Amdahl's Law	Gustafson's Law
Problem size	Fixed as cores increase	Grows with number of cores (run time roughly constant)
Result	Speedup limited by serial fraction: max 1/S	Scaled speedup grows linearly with N
Outlook	Pessimistic about massive parallelism	More realistic view of parallel computing on multi-core
Origin	Gene Amdahl, 1967	Sandia National Lab 1988 results (Gustafson); formula by E. Barsis

1.8 Chapter 1 key points (as given in the book)

- **Concurrency**: multiple threads in progress at the same time, often achieved on sequential processors through **interleaving**.
- **Parallelism**: multiple threads executing simultaneously.
- Modern applications often consist of multiple processes or threads that can execute in parallel.
- Most modern platforms are **MIMD** machines.
- In practice **Amdahl's Law does not accurately reflect** the benefit of increasing cores; **linear speedup is achievable by expanding the problem size** with the number of cores.

PART B: CHAPTER 2 - System Overview of Threading

2.1 Introduction

- Threading, implemented properly, enhances performance by making better use of hardware. Improper use causes **degraded performance, unpredictable behaviour, and hard-to-resolve errors**.
- The chapter presents threading from the **hardware up** through the **operating system** to the **application level**.

To understand threading for your application you need to understand:

- The **design approach and structure** of the application
- The **threading API**
- The **compiler or runtime environment**
- The **target platforms** on which it will run

From these a **threading strategy** is formulated for specific parts of the application. Threading can be simple once the basic principles are grasped.

2.2 Defining threads

Definitions

- **Thread:** a **discrete sequence of related instructions** that is executed **independently** of other instruction sequences.
- Every program has at least one thread: the **main thread**, which initialises the program and begins executing initial instructions. It may create other threads or do all the work itself.
- Each thread maintains its **current machine state**.
- At the **hardware level**, a thread is an execution path that remains independent of other hardware thread execution paths. The OS maps software threads to hardware execution resources.

- Not everything should be threaded; **too much threading can hurt performance**. Thoughtful design and proper testing determine the right balance. The decision should reflect the needs of the program and the execution capabilities of the deployment platform.

2.3 System view of threads: the three layers

Layer	Description
User-level threads	Threads created and manipulated in the **application software**.
Kernel-level threads	The way the **operating system** implements most threads.
Hardware threads	How threads appear to the **execution resources in the hardware**.

- A single program thread frequently involves **all three levels**: implemented by the OS as a kernel-level thread and executed as a hardware thread.
- Between the layers are **interfaces**, frequently handled automatically by the executing system, but it is important to know how they work.

A. Threading above the operating system

- Without a runtime framework, thread creation code is a **call to system APIs**; at runtime these become **calls to the OS kernel** to create a thread; the thread's instructions are passed to the processor.

Thread flow in a traditional application (Figure 2.2):

Stage	Performed by	What happens
Defining and Preparing Threads	Programming environment and compiler	Threads are specified by the programming environment and encoded by the compiler.
Operating Threads	OS (using processes)	Threads are created and managed by the operating system.
Executing Threads	Processors	Processor executes the sequence of thread instructions. (After execution, operations return to user space.)

- **Managed code** runs in a runtime environment that performs some application functions and calls the underlying OS. Examples: **Java Virtual Machine (JVM)** and Microsoft's **Common Language Runtime (CLR)**. These **do**

not provide scheduling; they rely on the **OS scheduler**.

Threading APIs available at the application level

- The most common: **OpenMP** and explicit low-level threading libraries such as **Pthreads** and **Windows threads**. Choice depends on requirements and platform.

	Low-level threading (Pthreads, Windows threads)	OpenMP
Code needed	Significantly **more code**	Less code
Benefit	**Fine-grained control** over thread use	**Ease of use**, more developer-friendly
Requirement	Only access to the OS multi-threading libraries	A **compiler that supports the OpenMP API** (limited to C/C++ and Fortran)
Thread creation	Explicit: e.g. `pthread_create()` creates a single thread and points it at the work function	Automatic: threads created in the background; no thread-creation function visible

Hello World examples (Listings 2.1 and 2.2):

```
// OpenMP version (Listing 2.1) - key parts
#pragma omp parallel private(threadID)
{
    threadID = omp_get_thread_num();
    printf("\nHello World is from thread %d\n", (int)threadID);
    if (threadID == 0) {                /* master thread has ID 0 */
        printf("\nMaster thread being called\n");
        totalThreads = omp_get_num_threads();
        printf("Total number of threads are %d\n", totalThreads);
    }
}

// Pthreads version (Listing 2.2) - key parts
rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
if (rc) { printf("ERROR return code from pthread_create(): %d\n", rc); exit(-1); }
...
pthread_exit(NULL);
```

- OpenMP: `#pragma omp parallel private(threadID)` marks a parallel block; the **threadID is private**; the master thread has threadID = 0. Must `#include <omp.h>`.
- Pthreads: includes `<pthread.h>`; defines `NUM_THREADS 5`; loop creates 5 threads each running `PrintHello()`; return code checked for errors.

B. Threads inside the OS

- The OS is partitioned into **two layers**: the **user-level partition** (where applications run) and the **kernel-level partition** (where system-oriented activities occur).
- **System libraries** = the interface between the application layer and the kernel-level OS; contain OS components that can run with **user-level privilege**.
- **HAL (Hardware Abstraction Layer)** = the interface between the operating system and the processor.
- Kernel-level OS contains: process, thread and resource scheduler; I/O manager; memory manager; kernel internal operational manager; other operational units.

Kernel-level vs user-level threads

- The **kernel** is the nucleus of the OS and maintains **tables** to keep track of processes and threads. Instead of a per-process thread list, the kernel maintains a **thread table** for all threads.
- The vast majority of thread-level activity relies on **kernel-level threads**. OpenMP and Pthreads use kernel-level threads.
- **Windows** supports both kernel-level and user-level threads. **User-level threads on Windows are called fibers**. They require the programmer to create the entire **management infrastructure** and **manually schedule** execution. Benefit: developer can manipulate details obscured in kernel threads. But because of manual overhead and limitations, fibers may add little value for well-designed multi-threaded apps.
- **Kernel-level threads** give better performance; multiple kernel threads of the same process can execute on **different processors or cores**. Overhead of kernel threading is higher than user-level threading, so kernel threads are frequently **reused** after finishing their work.
- User-level threads (e.g. fibers) are **mapped to kernel threads**; when executing, the processor knows them only as kernel-level threads.

Processes and threads

- **Process:** a discrete program task with its **own address space**; the coarse-level execution unit maintained as an independent entity inside the OS.
- Multiple threads can reside in one process. **All threads in a process share the same address space**, so they benefit from simple **inter-thread communication**.
- The OS assigns each process a **Process Control Block (PCB)** containing: the process's **unique identity**, **current machine state**, **priority**, and the **address of the virtual memory** where the process resides.
- A program has one or more processes, each containing one or more threads, each **mapped to a processor by the OS scheduler** (Figure 2.4).
- **Processor affinity:** lets the programmer **request** mapping of a specific thread to a specific processor. Most OSs attempt to obey but **do not guarantee** it.

Thread-to-processor mapping models (Figure 2.5)

Model	Who schedules	Name / notes
One to one (1:1)	**OS** handles thread scheduling; **no thread-library scheduler overhead**.	Also called **preemptive multi-threading**. Used by **Linux, Windows 2000, Windows XP**. Enables stronger OS handling of threads; the book focuses on this model (Windows and Linux).
Many to one (M:1)	The **library scheduler** decides which thread gets priority.	Called **cooperative multi-threading**.
Many to many (M:N)	Flexible mapping.	Mapping is flexible.

C. Threads inside the hardware

- Hardware executes the instructions from the software levels. Instructions of application threads are mapped to resources and flow down through the OS, runtime environment and virtual layer to hardware.
- Earlier, parallelism required **multiple CPUs** (each thread on its own processor). Today **HT Technology and multiple cores** provide multi-threading on a single processor.
- A CPU with only **one execution engine/core** but sharing pipeline and other resources among threads is **concurrent but not parallel** (Figure 2.6).

Hardware threading terms

- **Multi-core (CMP/CMT):** two or more execution cores = **true hardware-based multi-threading**. Because both threads execute on the **same processor**, the design is sometimes called **chip multi-threading (CMT)**.
- **HT Technology (SMT):** a **single core** in which two threads **share most of the execution resources**. Called **simultaneous multi-threading (SMT)**. SMT uses a **hardware scheduler** to manage different hardware threads that need resources.
- For **true parallelism**, the number of active program threads should equal the number of available hardware threads. In most cases program threads exceed hardware threads, but **too many software threads can slow performance**; keep a **balance**.

2.4 What happens when a thread is created

- A process can have more than one thread; each operates independently though they **share the same address space and certain resources such as file descriptors**.
- Each thread needs its **own stack space**, usually managed by the OS (Figure 2.7: program code + data, heap, and one stack region per thread).
- Application developers usually need not worry about stack details (stack sizes, allocation); **system-level developers must**. You must be aware of the OS limits; sometimes you must bypass the default stack manager and manage stacks yourself.
- **Default stack size varies** from system to system; that is why creating many threads on some systems can **slow performance dramatically**.
- Every process has **at least one thread**, the **initial thread**, created as part of process initialisation. Threads you create run while the initial thread continues. At program termination, the main thread and subsidiary threads are terminated.

Thread states (Figure 2.8)

EXAM FAVOURITE: four basic states

- A thread is always in one of four states: **Ready, Running, Waiting (Blocked), Terminated**.
- There are additional **sub-states** for various reasons; they are valuable for debugging/analysing threaded applications.
- **Each created thread starts in the Ready state.** When it attempts to execute instructions it is either **Running** or **Blocked**. It is blocked if waiting for a **resource** or for **another thread**.
- When a thread completes its work it is either **Terminated** or put back into **Ready** by the program.

Transition (from the state diagram)	Meaning
New -> Ready (Enter)	Thread created and enters the ready state.
Ready -> Running (Scheduler Dispatch)	Scheduler picks the thread to run.
Running -> Ready (Interrupt)	Thread is interrupted/pre-empted and goes back to ready.
Running -> Waiting (Event Wait)	Thread waits for an event or resource.
Waiting -> Ready (Event Completion)	Awaited event completes.
Running -> Terminated (Exit)	Thread finishes.

2.5 Application programming models and threading

- Threads are used liberally by the OS for its own activities, so even a **single-threaded application runs in a heavily threaded runtime setup**.
- All major languages support threads: **imperative** (C, Fortran, Pascal, Ada), **object-oriented** (C++, Java, C#), **functional** (Lisp, Miranda, SML), **logical** (Prolog).

2.6 Virtual environment: VMs and platforms

Virtualization

- **Virtualization** = using computing resources to create the **appearance of a different set of resources**.
- **Runtime virtualization** (e.g. Java JVM): creates the appearance that a Java application is running in its own private environment/machine.
- **System virtualization:** creates the appearance of a different kind of virtual machine with a **complete and independent instance of the operating system**.
- Both make effective use of threads internally.

Runtime virtualization

- Provided by a **runtime virtual machine**: a container and executor application on top of the OS. Two mainstream VMs: the **Java VM** and Microsoft's **Common Language Runtime (CLR)**.
- These VMs create **at least three threads**: (1) the **executing thread**, (2) a **garbage-collection thread** that frees memory blocks no longer in use, (3) a thread for **just-in-time (JIT) compilation** of bytecodes into executable binary code. They generally create other threads for internal tasks.

- The VM and the OS work in tandem to map these threads to the available execution resources.

System virtualization

- VMs recreate a **complete execution context**: virtualized network adapters and disks, own OS instance. **Several VMs** run on one hardware platform, each with its own OS.
- The virtualization layer between the host system and VMs is the **Virtual Machine Monitor (VMM)**, also known as the **hypervisor** (Figure 2.9: without VMs a single OS owns all hardware; with VMs multiple OSes share hardware).
- A VMM delivers virtualization such that the OS in each VM runs under the **illusion that it owns the entire hardware platform**.

Virtual processors and privileged instructions

- VM software uses **virtual processors**. It presents **as many virtual processors to the guest OS as there are cores** on the host hardware. **HT Technology does not change the number of virtual processors, only the core count.**
- Benefit: **isolation of the instruction-set architecture (ISA)**. Certain instructions are **privileged** and executable only by the OS. On Intel processors only the **host OS** has this privilege; the **VMM and the entire VM run as applications**.
- When a guest OS needs a privileged instruction: it is **trapped** by the virtual processor and a call is made to the VMM. The VMM may handle it itself, or pass it to the host OS, wait for the response and **emulate that response** in the virtual processor.
- This has a **performance cost**, so Intel developed ISA extensions, part of **Intel Virtualization Technology**, giving efficient ways for VMMs to execute guest OS privileged instructions.

Mapping application threads

- VMMs do very little unusual to handle application threads. **Thread creation and scheduling are handled by the guest OS**; the VMM need not know. The virtual processor executes the instructions like any other sequence.
- The VMM makes **no attempt to match application threads to specific processor cores** or to second-guess the guest OS's scheduler. Example: on a dual-core, the VMM presents **two virtual processors** to the guest OS, which schedules threads as on real hardware.

Lock-holder preemption (common trap)

- The VMM interrupts normal flow only when it must **swap out a VM** or perform internal tasks. With multiple guest VMs it must **time-slice** between them.
- Suppose a thread is **locked and waiting** for a thread on a different virtual processor, and that other processor is **swapped out**. The original VM sees a **substantial delay** that would not occur on dedicated hardware.
- This problem is called **lock-holder preemption**. It arises because guest VM resources must sometimes be swapped out and thread states may not expect this. The book expects OSs to offer features helping VMMs coordinate this as virtualization spreads.

2.7 Chapter 2 key points (as given in the book)

- Threading concepts depend on understanding the **interaction of various system components**; understand the impact of threads on system components.
- **Software threads differ from hardware threads** but maintain a direct relationship.
- Application threading can be implemented using **APIs or multi-threading libraries**.
- **Processes, threads and fibers** are different levels of the execution mechanism in a system.
- The thread life cycle has four stages: **ready, running, waiting (blocked), terminated**.
- Two types of virtualization: **runtime virtualization** and **system virtualization**.
- A **VMM typically makes no attempt to match application threads to specific processor cores** or to second-guess the guest OS's scheduler.

PART C: QUICK REVISION SHEET

Formula sheet

Name	Formula	Notes
Speedup	Time(best sequential) / Time(parallel, n_t)	n_t = number of physical threads
Amdahl (general)	$1 / [(1 - F_{enh}) + F_{enh}/Speedup_{enh}]$	Half program sped up 15% gives 1.08
Amdahl (Eq 1.1)	$1 / [S + (1 - S)/n]$	S = serial time fraction; n = cores
Upper bound (Eq 1.2)	$1 / S$	10% serial gives max 10x
With overhead	$1 / [S + (1 - S)/n + H(n)]$	H(n) = OS + inter-thread overhead; can make speedup < 1
HT Amdahl	$1 / [S + 0.67((1 - S)/n) + H(n)]$	n = logical processors; each thread ~1/3 slower
Gustafson (scaled)	$N + (1 - N) * s$	s = serial fraction; linear speedup

Worked-number cheat list

- Dual-core, half work parallel: **1.33**. 8-core, half work parallel: **1.78**.
- 30% parallel, dual-core: **85%** run time. Double cores: **77.5%**. Double parallel code: **70%**.
- HT gain: about **30%** (1.3x). Multi-core dual: up to **2x**. Quad-core: up to **4x** in theory.
- Moore's law: doubling every **18 to 24 months** (1965). Flynn's taxonomy: **1972**. Amdahl: **1967**. Gustafson/Sandia: late **1980s**, **1,024** processors. EDVAC memo: **1945**. Time-sharing OS: **1960s**.

Comparison tables

	Concurrent	Parallel
Hardware	Single hardware resource, interleaved	Different hardware resources / processing elements
Progress	One thread at a time	Multiple threads simultaneously
Example	Time-slicing, HT (shared core)	Multi-core, multiprocessor

	Single-core threading	Multi-core threading
Purpose of threads	Hide latency, responsiveness	Partition workloads for real speedup
Execution	Interleaved only	Simultaneous on separate cores
Cache	One cache shared; no sync problem	Each core may have its own; **false sharing** possible
Thread priority assumption	Higher priority runs uninterrupted: may hold	May fail; both may run at once

	User-level threads	Kernel-level threads
Created by	Application software / library	OS
Scheduling	Manual / by library (cooperative)	OS scheduler (preemptive)
Windows name	Fibers	Windows threads
Overhead	Lower, but developer must build management	Higher; threads often reused
Parallelism	Mapped onto kernel threads	Different kernel threads can run on different cores

Topic	One-line memory hook
Thread	Discrete sequence of related instructions executed independently

Topic	One-line memory hook
Process	Own address space; threads inside it share the address space
PCB	Identity, machine state, priority, virtual memory address
Affinity	Request, not guarantee
1:1 / M:1 / M:N	Preemptive (OS) / cooperative (library) / flexible
Thread states	Ready, Running, Waiting (Blocked), Terminated; new threads start Ready
VM threads	Executing + garbage collection + JIT (at least three)
VMM / hypervisor	Layer between host and VMs; virtual processors = number of cores; HT does not change count
Lock-holder preemption	Waiting thread delayed because the thread it waits for is on a swapped-out virtual processor
CMT vs SMT	Multi-core (true parallel) vs HT (shared core)

PART D: PRACTICE QUESTIONS WITH MODEL ANSWERS

Short answer (2 to 3 marks)

Q. Define concurrency in software.

A way to manage the sharing of resources used at the same time.

Q. Distinguish concurrent from parallel execution.

Concurrent: thread execution is interleaved onto a single hardware resource; threads are ready but only one makes progress at a time. Parallel: threads run simultaneously on different hardware resources/processing elements. Parallelism requires concurrency exploiting multiple hardware resources.

Q. State Moore's law. Is it a prediction of clock frequency?

Observed by Gordon Moore in 1965: the number of transistors available to manufacturers doubles about every 18 to 24 months. It is not really a predictor of clock frequency, though clock speed has tended to follow it; the extra transistors can be used in other ways such as ILP.

Q. What is ILP?

Instruction-level parallelism, also called dynamic or out-of-order execution: the CPU reorders instructions to eliminate pipeline stalls and execute more instructions per clock cycle. It is done by the processor and is transparent to the software developer.

Q. What is a logical processor?

Created by duplicating the architecture state (general-purpose registers and interrupt controller registers) of a physical processor while sharing execution resources; makes one processor appear as multiple processors to software (SMT/HT).

Q. What is false sharing?

Two threads on different cores access independent but neighbouring memory locations in the same cache line. The line is marked invalid in the other core's cache even though the needed data has not changed, causing needless cache synchronisation traffic.

Q. Name the four states of a thread.

Ready, Running, Waiting (Blocked), Terminated.

Q. What is processor affinity?

The ability of the programmer to request that a specific thread be mapped to a specific processor. Most OSs try to obey but do not guarantee it.

Q. What is a VMM?

Virtual Machine Monitor, also known as the hypervisor; the virtualization layer between the host system and the virtual machines, making each guest OS believe it owns the entire hardware platform.

Q. What is lock-holder preemption?

A thread is waiting for a lock held by a thread on another virtual processor that has been swapped out by the VMM, causing a substantial delay not seen on dedicated hardware.

Medium answer (5 marks)

Q. Explain Flynn's taxonomy with examples.

Computers are classified by number of instruction streams and data streams: SISD (traditional sequential, e.g. original IBM PC, Commodore 64), MISD (mostly theoretical), SIMD (one instruction on multiple data; vector processors like Cray-1; Intel MMX/SSE/SSE2/SSE3; PowerPC AltiVec; used in DSP, image, audio/video), MIMD (multiple instruction streams on separate data streams; most common today; multi-core platforms like Intel Core Duo).

Q. Why is concurrency important in software? Give examples.

(1) Efficient use of resources: a serial streaming client (wait for data, uncompress, decode, send to hardware) leaves the system idle while waiting; staging the work lets the previous frame be decoded while the next arrives. (2) Many problems are naturally concurrent: in an FTP server a thread per connection manages its own state, which is simpler than one thread tracking all connections.

Q. Differentiate Hyper-Threading Technology from multi-core.

HT shares one core's execution engine between two threads and interleaves instructions, hiding latency; no true parallelism; about 30% throughput gain in certain applications (1.3x); may give none if latencies were already removed by cache optimisation. Multi-core has two or more independent execution cores, each thread has its own hardware environment, true parallel execution, theoretically near 2x for dual-core. HT does not deliver multi-core performance but combined with multi-core gives more throughput.

Q. What differences must a developer keep in mind between multi-threading on single-core and multi-core?

(1) Single-core threads only interleave so gains are limited and threads mainly hide latency; on multi-core threads run independently on separate cores and workloads can be partitioned. (2) Memory caching: separate caches per core can cause false sharing; single-core has one cache. (3) Thread priority: assuming a high-priority thread runs uninterrupted is valid on single-core but both may run simultaneously on multi-core, making such code unstable.

Q. Explain the three layers of threading.

User-level threads are created and manipulated in application software; kernel-level threads are how the OS implements most threads; hardware threads are how threads appear to the execution resources. A single program thread usually involves all three: it becomes a kernel-level thread and runs as a hardware thread. Interfaces between layers are frequently automatic.

Q. Compare user-level and kernel-level threads.

User-level threads (fibers on Windows) need the programmer to build management infrastructure and schedule manually; they give control over details but add overhead and limitations. Kernel-level threads are managed by the OS, give better performance, and threads of one process can run on different cores. Kernel threads have higher overhead than user threads so are reused after finishing their work. User threads map onto kernel threads, so the processor only sees kernel threads.

Q. Describe the thread-to-processor mapping models.

1:1: OS schedules each thread, no library scheduler overhead, preemptive multi-threading, used by Linux, Windows 2000/XP. M:1: library scheduler decides priority, cooperative multi-threading. M:N: flexible mapping.

Q. Explain what happens when a thread is created, including its life cycle.

The thread shares the process address space and resources such as file descriptors but gets its own stack managed by the OS; default stack size varies, so many threads can slow performance dramatically. It starts in Ready; the scheduler dispatches it to Running; it may be Blocked when waiting for a resource or another thread, goes back to Ready after the event or an interrupt, and finally is Terminated or returned to Ready by the program.

Q. Explain runtime and system virtualization and how threads are handled.

Runtime virtualization (JVM, CLR) gives an application the appearance of its own private machine; VMs create at least an executing thread, a garbage-collection thread and a JIT thread, and the VM and OS map them to hardware. System virtualization (VMM/hypervisor) creates VMs with complete OS instances; virtual processors equal the number of cores (HT does not change this); privileged instructions of guests are trapped and emulated by the VMM, a cost reduced by Intel Virtualization Technology; application threads are scheduled entirely by the guest OS and the VMM does not match them to cores.

Long answer / numerical (8 to 10 marks)

Q. State Amdahl's Law, derive the upper bound, and discuss its corollary and the effect of overhead.

Speedup = 1/[S + (1-S)/n], S = serial fraction, n = cores. With n tending to infinity speedup = 1/S, so with 10% serial the max is 10x regardless of core count. Corollary: reducing the serial portion matters more than adding cores; e.g. 30% parallel on dual-core: doubling cores moves run time from 85% to 77.5%, but doubling parallel code moves it from 85% to 70%. With overhead: Speedup = 1/[S + (1-S)/n + H(n)], H(n) includes OS overhead and synchronisation/communication; if large, speedup can fall below 1, meaning threading slowed the program. Hence threading overhead must be minimised.

Q. A program is 50% parallelisable. Compute speedup on 2 and 8 cores.

S = 0.5. n = 2: 1/(0.5 + 0.25) = 1/0.75 = 1.33. n = 8: 1/(0.5 + 0.0625) = 1/0.5625 = 1.78 (matching the book's results).

Q. What is the maximum speedup if 10% of an application is serial?

1/S = 1/0.10 = 10x, regardless of number of cores.

Q. Half of a program is sped up by 15%. Find the overall speedup.

1/[(1-0.5) + 0.5/1.15] = 1/(0.5 + 0.43) = 1.08, an 8% increase.

Q. Discuss Gustafson's Law and why it is more realistic than Amdahl's.

Late 1980s Sandia results on a 1,024-processor hypercube showed near-linear speedups in practical applications despite Amdahl's predictions. Amdahl's Law assumes (1) the best serial algorithm is limited only by CPU cycles (ignoring per-core caches reducing memory latency), (2) the serial algorithm is the best possible (some problems have more efficient parallel solutions), and (3) a fixed problem size as cores increase, which is its biggest weakness, since problems grow to fit available resources and run time is often constant. Scaled speedup (E. Barsis) = N + (1-N)s, linear in N; shown equivalent to Amdahl's by Shi (1996) but gives a more realistic view of multi-core potential.

Q. Give the Amdahl form for Hyper-Threading and explain the 0.67 factor.

Speedup(HTT) = 1/[S + 0.67((1-S)/n) + H(n)], n = logical processors. Shared resources make each thread run about one-third slower than with the entire processor, so the parallel term is scaled by 0.67. H(n) is determined empirically per application. HT is not a substitute for multi-core.